Can a Frit Stop a Runaway?

A Validated Stability Simulator and AI Frit Designer
for Particle Bed Nuclear Rocket Reactors

Project Outline for SwampHacks XII (Software Edition)

University of Florida

[TO DO: Team name]
[TO DO: Team members]

Status: planning document. Nothing has been built or computed yet. All results tables are blank. Every result this project produces is a **model prediction** from a physics simulation, never a measurement, and is labeled that way throughout. What keeps it from being hardcoded is explained in Section 2.

Contents

1 The Project on One Page **2**

2 Why This Is Not Hardcoded **3**

3 How This Project Scores on Each Criterion **3**

4 Introduction **4**

4.1 What a particle bed reactor is . . . 4

4.2 The problem: flow instability and flow shift . . . 4

4.3 What this project does . . . 4

5 Research Question and Relevance **5**

5.1 Research question . . . 5

5.2 Sub-questions . . . 5

5.3 Hypotheses (from the literature, not from results) . . . 5

5.4 Why it matters scientifically . . . 5

5.5 Why it matters to everyday people . . . 6

6 Scope Tiers and Time Budget **6**

6.1 Tier 1: must-have (complete working project) . . . 6

6.2 Tier 2: should-have (the AI layer) . . . 6

6.3 Tier 3: stretch . . . 7

6.4 Time budget (36 hours) . . . 7

6.5 Suggested roles (team of 2–4) . . . 7

7 Methods **8**

7.1 Model overview . . . 8

7.2 Parameters (all from sources) . . . 9

7.3 Physics in each path . . . 9

7.4 Steady state and transient . . . 10

7.5 Verification (is the code solving the equations correctly?) . . . 10

7.6 Validation (does the model agree with independent results?) . . . 10

7.7 Studies . . . 11

7.8 AI methods . . . 11

7.9 The web app . . . 11
7.10 Software: what you need to code . . . 12

8 Risks and Fallbacks **13**

9 Data **14**
9.1 Where each kind of data comes from . . . 14
9.2 Data management . . . 14
9.3 Results tables (fill in only with solver output) . . . 14
9.4 Figures to produce . . . 15

10 Data Analysis **16**
10.1 Metrics . . . 16
10.2 Analysis steps . . . 16
10.3 Limitations to state openly . . . 16

11 Pitch, Demo, and Q&A **17**
11.1 Three-minute demo script . . . 17
11.2 Prepared answers to hard questions . . . 17
11.3 Devpost checklist . . . 18

12 Conclusions (To Be Written) **18**
12.1 Future work . . . 18

13 Full Pipeline **18**

Sources **19**

Acknowledgements **20**

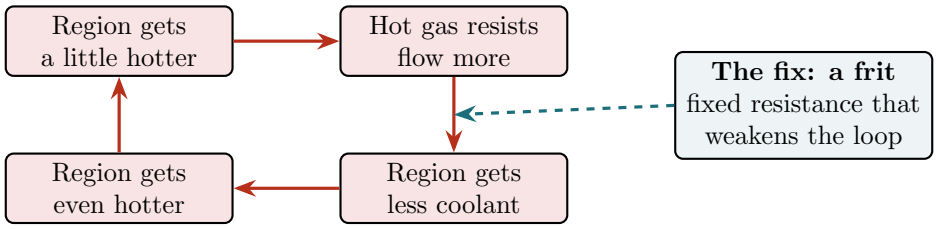
1 The Project on One Page

One-sentence pitch. When parallel channels share one coolant supply, the hottest channel can starve itself and run away, a known risk for particle bed nuclear rocket reactors, so we built a physics simulator checked against NASA and published results, plus an AI that designs reactor frits to keep the core stable, and judges can try to break it themselves.

What judges will see

A web app showing one particle bed fuel element. A judge sets reactor power, frit resistance, and flow pattern. The physics solver runs live and shows the coolant redistributing and the fuel temperatures across the bed. With no cold frit at high power, a hot spot runs away on screen. With a frit, the same power stays stable. Next to the solver's answer, the AI's instant prediction appears, along with an AI-designed frit that outperforms a uniform one. A validation tab shows how the model compares with published results.

The feedback loop, in plain words



Glossary for judges

Term	Plain meaning
Particle bed reactor	A nuclear rocket design where hydrogen flows through a bed of tiny fuel beads and is heated to make thrust.
Frit	A porous wall around the fuel bed that adds a fixed amount of flow resistance.
Flow instability	The runaway loop above: the hottest region loses its coolant.
Validation	Checking the simulator against results it was not tuned to match.
Surrogate model	An AI model trained on many simulator runs that predicts results instantly.
Bayesian optimization	An AI search method that picks which designs to try next to find the best one quickly.

2 Why This Is Not Hardcoded

A simulation is hardcoded when its answers are decided by the person who wrote it. This project prevents that in five ways:

1. **Results come from solving equations.** Flow split, temperatures, and stability are computed from conservation of mass, momentum, and energy, with real hydrogen properties from a database. No outcome is written into the code.
2. **Inputs are fixed from sources before checking results.** Every geometry and operating value is taken from the literature and recorded with its source in `params.py`, which is committed to Git before any validation run.
3. **Validation against independent results.** The model is compared with published conclusions and a textbook stability criterion it was not tuned to match. Agreement criteria are written down before running. Mismatches are reported, not hidden.
4. **AI is tested on data it never saw.** The surrogate model's accuracy is reported on held-out simulations and on a region outside its training range.
5. **Judges choose the inputs.** The app runs the physics solver live on whatever settings a judge picks, so nothing can be pre-scripted.

> **What this project cannot claim.** It produces model predictions, not experimental evidence. Its strongest claims depend on how well the validation turns out, and it says so.

3 How This Project Scores on Each Criterion

Criterion	How the project addresses it	Evidence to show judges
Technical Execution	A multi-path thermal-hydraulic solver with real hydrogen properties, stability analysis, verification tests, an AI surrogate, and an optimizer.	Live solver in the app; passing verification tests; Git history from the event.
Innovation & Creativity	AI-designed frits tested on an open question (cold versus hot frit control) with a judge-driven simulator.	"Try to break it" demo; AI design versus uniform frit.
Impact & Purpose	A known open risk for nuclear rockets bound for Mars; the same physics as computer and data-center cooling; an open-source teaching tool.	Pitch; stability map; "Why it matters" slide.
Clarity & Communication	Plain-language pitch, feedback-loop diagram, scripted demo, prepared answers.	Section 11.
Completion & Scope	No hardware risk; Tier 1 is a complete working app by hour 24.	Tier 1 checklist; backup video.

Trade-off: without hardware, the project gives up the physical "wow" and the hardware prize. It makes up for that with a live, interactive app that judges control, and with more time for polish.

4 Introduction

4.1 What a particle bed reactor is

A particle bed reactor (PBR) is a nuclear thermal propulsion (NTP) concept: a reactor heats hydrogen, and the hot hydrogen expands through a rocket nozzle. In each fuel element, small spherical fuel particles are packed randomly in an annular region between two porous cylinders, the **cold frit** (outside) and the **hot frit** (inside). Hydrogen flows radially inward through the cold frit and the bed, then leaves axially through the central channel inside the hot frit. The PBR was the basis of the U.S. Space Nuclear Thermal Propulsion (SNTP) program.

4.2 The problem: flow instability and flow shift

Because coolant in a particle bed has no fixed channels, it can redistribute itself. Heating hydrogen raises its viscosity and lowers its density, so a hotter region resists flow more, creating the feedback loop in Section 1.

A NASA analysis asked whether flow through the bed would be well behaved or subject to destructive instabilities. It noted that most stability analyses treat the bed as an array of parallel passages (the approach of Bussard and DeLauer). It concluded that a particle bed *without* a cold frit would be subject to instability at the high temperatures desired for nuclear rockets, and that an appropriate cold frit could very likely cure it.

Researchers at Tsinghua University describe a "flow shift" that produces a hot spot in the core, and propose reducing it by lowering the pressure drop in the hot channel, adding resistance at the cold or hot frit, or changing the flow pattern from "Z" to "U." They note there was no consensus on whether the cold or the hot frit should control the flow.

4.3 What this project does

This project builds a parallel-passage simulator of one PBR fuel element, checks it against the NASA and Tsinghua conclusions and a textbook stability criterion, and uses AI to (1) predict stability instantly and (2) design frit resistance profiles that keep the element stable at higher power with low pressure drop. A live web app lets judges explore it.

5 Research Question and Relevance

5.1 Research question

> In a validated parallel-passage model of a particle bed fuel element, how much frit resistance, and on which frit, is needed to prevent flow instability as power rises? Can an AI-guided search find frit designs that are stable at higher power with less pressure drop than conventional designs?

5.2 Sub-questions

1. Does the model reproduce NASA's conclusion (unstable without a cold frit at high temperature, cured by a frit) without being tuned to it?
2. Does it reproduce the Tsinghua trends for Z versus U flow and added frit resistance?
3. Is flow distribution better controlled by the cold frit or the hot frit in this model?
4. How accurately does an AI surrogate predict stability on cases it never saw?
5. Does Bayesian optimization beat uniform, hand-designed, and random frit designs?

5.3 Hypotheses (from the literature, not from results)

- **H1:** without a cold frit, the element becomes unstable above some outlet temperature.
- **H2:** more cold frit resistance raises the maximum stable power.
- **H3:** the U flow pattern reduces flow nonuniformity compared with Z.
- **H4:** which frit controls flow better is an open question; the model tests it rather than assuming an answer.

5.4 Why it matters scientifically

- **An unresolved design question.** Whether flow through a particle bed is well behaved was identified as an unresolved question for the concept, and frit design is the main proposed fix.
- **An open disagreement.** Published studies disagree on whether the cold or hot frit should control flow.
- **Space exploration.** NTP could at least double the performance of the best chemical rocket engines, and NASA's Mars Design Reference Architecture 5.0 selected it as the leading candidate for a crewed Mars mission.
- **Open tools.** A validated, open-source simulator with an AI design layer can be reused for teaching and for early design studies.

5.5 Why it matters to everyday people

- **Computers and AI data centers.** Servers share cooling air; if one path gets less airflow, it runs hotter. The same uneven-flow physics applies, and cooling is a growing cost of AI computing.
- **The kitchen.** Uneven flow through packed particles is called "channeling." It is why an espresso shot can taste sour and bitter at once.
- **Shared supply, uneven demand.** Building ducts, water pipes, and boiler tubes all split one flow among parallel paths; adding fixed resistance to each path is a common way to keep them stable.
- **Faster trips to Mars.** Better nuclear rocket engines mean shorter trips and less time astronauts spend exposed to space radiation.

6 Scope Tiers and Time Budget

> **Rule:** do not start Tier 2 until every Tier 1 item works and the backup video is recorded.

6.1 Tier 1: must-have (complete working project)

- ☐ Parameters collected from sources and frozen in `params.py`.
- ☐ Steady-state multi-path solver with hydrogen properties, bed pressure drop, frits, and Z/U flow.
- ☐ Verification tests pass (Section 7.5).
- ☐ Transient model and stability analysis.
- ☐ Power sweep and frit sweep producing a stability map.
- ☐ Validation against NASA's conclusion and the textbook stability criterion.
- ☐ Web app with the live solver, temperature map, flow distribution, and runaway animation.
- ☐ Backup video, Devpost draft, public repository.

6.2 Tier 2: should-have (the AI layer)

- ☐ Training dataset from the solver; surrogate model with held-out and out-of-range tests.
- ☐ "AI versus physics" panel in the app.
- ☐ Bayesian optimization of frit profiles; best designs verified with the full solver.
- ☐ Cold versus hot frit comparison; Tsinghua Z versus U comparison.

6.3 Tier 3: stretch

- ☐ Active learning to map the stability boundary with fewer runs.
- ☐ Reactor power coupling: point kinetics with fuel temperature feedback.
- ☐ Local (azimuthal) hot spots within each axial slice.
- ☐ Uncertainty analysis over the literature ranges of uncertain parameters.
- ☐ Public hosting of the app.

6.4 Time budget (36 hours)

Hours	Work	Checkpoint
0–2	Repository, roles, environment; enter sourced parameters; freeze `params.py`.	Parameters committed.
2–8	Property wrapper, single-path model, multi-path steady solver.	Solver converges at nominal.
8–11	Verification tests.	All checks pass.
11–15	Transient model, stability analysis, power and frit sweeps.	Stability map exists.
15–17	Validation against NASA and the stability criterion.	Validation table filled.
17–22	Web app: inputs, live solver, plots, animation.	Judge can run it.
22–24	Polish; record backup video.	**Tier 1 complete.**
24–30	Tier 2: dataset (runs in background), surrogate, optimizer, comparisons.	Tier 2 complete or cut.
30–31	Tier 3 only if ahead.	Feature freeze at hour 31.
31–34	Figures, Devpost, README, final video.	Submission drafted.
34–36	Rehearse pitch and Q&A.	Pitch under 3 minutes.

6.5 Suggested roles (team of 2–4)

- **Physics model:** parameters, property wrapper, steady and transient solvers.
- **Verification and validation:** tests, literature comparisons, sweeps.
- **AI:** dataset, surrogate, optimizer.
- **App and communication:** web app, figures, Devpost, video, pitch.

7 Methods

7.1 Model overview

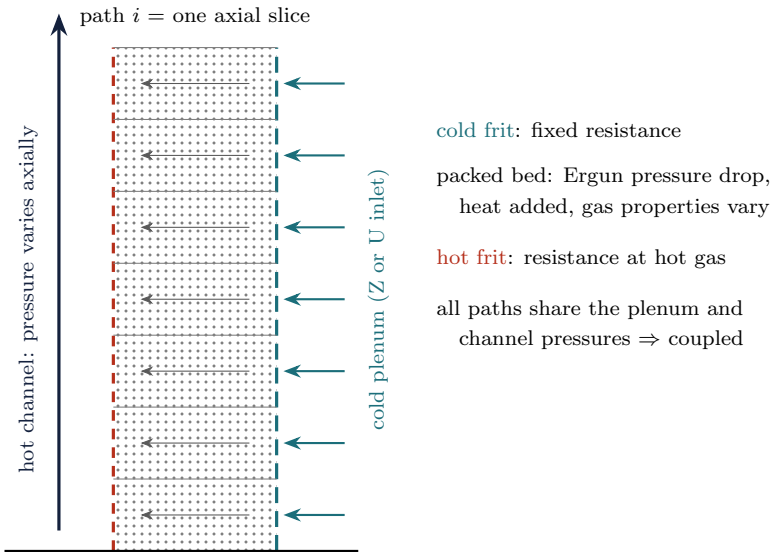


Figure 1: Parallel-passage model of one fuel element (half cross-section, not to scale). Each axial slice is a radial flow path from the cold plenum, through the cold frit, bed, and hot frit, into the hot channel.

The element is split into N parallel paths (start with $N = 20$ axial slices). Each path carries a radial mass flow $\dot{m}_i$. All paths share the cold plenum on the outside and the hot channel inside, so they are coupled through those pressures.

7.2 Parameters (all from sources)

Parameter	Source
Bed inner and outer radius, length	**[TO DO: SNTP report or Tsinghua papers]**
Fuel particle diameter, bed porosity	**[TO DO: source]**
Frit thickness, permeability, or loss coefficients	**[TO DO: source; use literature ranges]**
Inlet temperature and pressure; outlet temperature	**[TO DO: source]**
Power per element; axial power shape	**[TO DO: source]**
Fuel particle density and heat capacity	**[TO DO: source]**
Hydrogen viscosity, density, heat capacity, conductivity	CoolProp database (cross-checked with NIST)

Record every value with its citation in `params.py`. Where sources give a range, use the middle value as nominal and keep the range for the Tier 3 uncertainty analysis.

7.3 Physics in each path

Packed bed (Ergun equation, integrated radially). With superficial velocity u that grows as gas moves inward and heats up:

$$-\frac{dP}{dr} = 150\,\frac{\mu(T)\,(1-\varepsilon)^2}{\varepsilon^3 d_p^2}\,u + 1.75\,\frac{\rho(T,P)\,(1-\varepsilon)}{\varepsilon^3 d_p}\,u\,|u|, \qquad u = \frac{\dot{m}_i}{\rho\,2\pi r\,\Delta z_i}$$

Frits (porous walls). Each frit adds a Darcy–Forchheimer pressure drop evaluated at the local gas temperature:

$$\Delta P_{\text{frit}} = \frac{\mu}{k}\,t\,u + \beta_F\,\rho\,t\,u^2$$

A multiplier on each slice's frit resistance is the design variable for the optimizer.

Energy. Heat generated in the bed is carried away by the gas: $\dot{m}_i\,c_p\,dT_g = q'''\,2\pi r\,\Delta z_i\,dr$. Fuel and gas temperatures are linked by a packed-bed heat transfer correlation (Wakao and Kaguei): $\mathrm{Nu} = 2 + 1.1\,\mathrm{Re}^{0.6}\,\mathrm{Pr}^{1/3}$.

Hot channel and plenum. Pressure along the hot channel changes as flow from each slice accumulates (momentum and friction); the cold plenum is handled the same way. Feeding the plenum from the same end as the outlet gives the U pattern; from the opposite end gives Z.

Boundary condition. Fixed total pressure drop across the element (many elements share common plenums), with fixed total flow as an alternative case. **[TO DO: Decide which is primary and justify it.]**

7.4 Steady state and transient

- **Steady state:** solve for all $\dot{m}_i$ so every path's pressure drop matches the plenum and channel pressures (`scipy.optimize.root`, with continuation from low power to help convergence).
- **Transient:** fuel temperatures change slowly compared with gas flow, so treat gas flow as adjusting instantly and integrate fuel temperature in time: $C_i \, dT_{f,i}/dt = Q_i - hA\,(T_{f,i} - T_{g,i})$.
- **Stability:** at each steady state, compute the eigenvalues of the transient model's Jacobian; any eigenvalue with positive real part means unstable. Confirm with a small perturbation run that grows or dies out.
- **Textbook check (Ledinegg criterion):** a path fed at constant pressure difference is statically unstable where its pressure drop *decreases* as its flow increases. Compare this with the eigenvalue result.

7.5 Verification (is the code solving the equations correctly?)

Test	Pass condition
Constant-property bed	Matches the analytic Ergun result.
Mass conservation	Sum of path flows equals total flow.
Energy balance	Total heat equals hydrogen enthalpy rise.
Grid convergence	Key outputs change by less than **[TO DO: tolerance]** when paths and radial steps are doubled.
Symmetry	Uniform power and frits give equal path flows when channel pressure effects are switched off.
Property check	CoolProp values match NIST at spot temperatures.

Write these as automated tests (`pytest`) and run them after every change.

7.6 Validation (does the model agree with independent results?)

Write the agreement criterion for each check *before* running it.

Published result	Model test
NASA: unstable without cold frit at high temperature; frit cures it	Stability versus outlet temperature with zero and nonzero cold frit resistance.
Ledinegg criterion	Onset predicted by slope test versus eigenvalue analysis.
Tsinghua: U pattern reduces flow shift	Flow nonuniformity, Z versus U.
Tsinghua: added frit resistance reduces flow shift	Nonuniformity versus frit resistance.

7.7 Studies

ID		Study	What it does
S0	Tier 1	Verification	Run the tests above.
S1	Tier 1	Baseline element	Flow distribution and temperatures at nominal conditions.
S2	Tier 1	Power sweep, no cold frit	Find the power where instability begins; show a runaway transient.
S3	Tier 1	Frit sweep	Maximum stable power versus cold frit resistance (stability map).
S4	Tier 2	Z vs. U; cold vs. hot frit	Compare flow patterns and which frit controls flow better.
S5	Tier 2	AI dataset	Thousands of solver runs sampled across inputs (Latin hypercube).
S6	Tier 2	AI frit design	Bayesian optimization; verify best designs with the full solver.
S7	Tier 2	Blind tests	Pre-registered random inputs, plus judge-chosen inputs during demos.

7.8 AI methods

- Tier 2 **Surrogate model.** Inputs: power, axial power shape, frit resistance per slice (cold and hot), flow pattern, inlet conditions. Outputs: stability margin, peak fuel temperature, flow nonuniformity, total pressure drop. Start with a Gaussian process; use a small neural network if the dataset is large. Report accuracy on held-out runs and on an out-of-range test set.
- Tier 2 **Bayesian optimization.** Search frit profiles to maximize stable power while keeping pressure drop low, giving a set of best trade-offs. Baselines: uniform frit, a hand-designed taper, and random search with the same number of solver runs. Verify every reported design with the full solver.
- Tier 3 **Active learning** to choose solver runs near the stability boundary.
- Tier 3 **Point kinetics coupling** so reactor power responds to fuel temperature.

7.9 The web app

- **Controls:** power, cold frit resistance, hot frit resistance, flow pattern (Z/U), and "use AI-designed frit."
- **Live solver output:** temperature map of the bed, flow per slice, stability margin, and a transient animation after a small disturbance.
- **AI versus physics** (Tier 2): the surrogate's instant prediction next to the solver's result and its run time.
- **Tabs:** stability map, validation results, AI design comparison, "what's in the model."

- **Live mode** uses a coarser grid if needed for speed, and says so on screen. Never replace the live solver with precomputed answers.

7.10 Software: what you need to code

File	What it does
`params.py` **Tier 1**	All inputs with source citations; frozen before validation.
`properties.py` **Tier 1**	CoolProp hydrogen properties with a fast interpolation table.
`element.py` **Tier 1**	Geometry, slices, frit and bed pressure drop, heat transfer.
`steady.py` **Tier 1**	Multi-path flow-split solver with plenum and channel pressures.
`transient.py` **Tier 1**	Fuel temperature time integration; disturbance runs.
`stability.py` **Tier 1**	Eigenvalue analysis, Ledinegg check, maximum stable power search.
`tests/` **Tier 1**	Verification tests (`pytest`).
`validate.py` **Tier 1**	Literature comparisons with pre-written criteria.
`app.py` **Tier 1**	Streamlit web app.
`dataset.py` **Tier 2**	Input sampling and parallel solver runs.
`surrogate.py` **Tier 2**	Surrogate training and testing.
`optimize.py` **Tier 2**	Bayesian optimization and verification.
`analysis.py`	Figures and tables.

Libraries: `numpy`, `scipy`, `CoolProp`, `pandas`, `scikit-learn`, `matplotlib` or `plotly`, `streamlit`, `pytest`; optionally `torch` for a neural network.

Computer: a laptop is enough. Run the dataset generation in the background with several processes. The Raspberry Pi is not needed.

8 Risks and Fallbacks

Risk	Prevention	Fallback
Solver does not converge	Continuation from low power; damping; good initial guess.	Simplify to linear frit and bed resistances (still temperature-dependent), labeled as a simpler model.
Missing parameters	Collect sources before the event.	Use literature ranges; show results across the range.
No instability appears, or it appears everywhere	Check against the Ledinegg criterion.	Report it honestly as a finding; explain which assumption controls it.
Validation mismatch	Write criteria in advance.	Report and explain it; a well-explained mismatch is still credible.
App too slow for live use	Interpolation tables; coarse live grid.	Show a progress indicator; never swap in precomputed answers.
Surrogate inaccurate	Enough data; test out of range.	Report its errors; keep it as a demonstration and rely on the solver.
CoolProp install problems	Install and test beforehand.	Fit property tables to NIST data.
Running out of time	Tiers and checkpoints.	Cut Tier 3, then Tier 2; Tier 1 is complete on its own.
Judges say "it's just a simulation"	Section 2; validation tab.	Let a judge pick inputs and watch the solver run.

9 Data

9.1 Where each kind of data comes from

Type	Data	Role
Simulation results	Flow splits, temperatures, stability margins, pressure drops, AI training data	All results; always labeled as model predictions.
Database	Hydrogen properties (CoolProp; NIST for cross-checks)	Model inputs.
Literature	Geometry, operating conditions, correlations, published conclusions	Model inputs and validation targets.
Measured	None	This edition has no experiments.

9.2 Data management

- Every run saves its inputs, the Git commit of the code, and its outputs.
- Datasets saved with their random seeds so they can be regenerated exactly.
- `params.py` and validation criteria committed before validation runs.

9.3 Results tables (fill in only with solver output)

Verification

Test	Result	Pass?
Constant-property bed vs. analytic	_____	_____
Mass conservation error	_____	_____
Energy balance error	_____	_____
Grid convergence change	_____	_____

Validation

Published result	Model result	Agrees?
NASA: no cold frit unstable; frit cures	_____	_____
Ledinegg onset vs. eigenvalue onset	_____	_____
Tsinghua: U reduces flow shift	_____	_____
Tsinghua: frit resistance reduces flow shift	_____	_____

Stability map

Cold frit resistance	Max stable power	Pressure drop at that power
0 (none)	_____	_____
[TO DO: value]	_____	_____
[TO DO: value]	_____	_____

AI surrogate (Tier 2)

Test set	Stability margin error	Stable/unstable accuracy
Held-out (in range)	_____	_____
Out of range	_____	_____

Frit design (Tier 2)

Design	Max stable power	Pressure drop	Verified by solver?
Uniform frit	_____	_____	_____
Hand-designed taper	_____	_____	_____
Random search (best)	_____	_____	_____
Bayesian optimization, cold frit	_____	_____	_____
Bayesian optimization, hot frit	_____	_____	_____

9.4 Figures to produce

1. Model diagram.
2. Flow and temperature per slice at nominal conditions.
3. Runaway versus stable transient after the same disturbance.
4. Stability map: power versus frit resistance.
5. Validation comparison chart.
6. Surrogate predicted versus solver values, in range and out of range.
7. Pressure drop versus stable power for all frit designs, with the best trade-offs highlighted.

10 Data Analysis

10.1 Metrics

- **Flow nonuniformity:** coefficient of variation of slice flow fractions.
- **Peak fuel temperature** and its location.
- **Stability margin:** largest real part of the eigenvalues (negative is stable), and maximum stable power.
- **Total pressure drop** (pumping cost).
- **Surrogate error** and **classification accuracy** on held-out and out-of-range sets.
- **Optimization efficiency:** best design found versus number of solver runs.

10.2 Analysis steps

1. Confirm verification and grid convergence before using any result.
2. Fill in the validation table against the pre-written criteria.
3. Build the stability map and compare it with NASA's conclusion.
4. Compare Z with U and cold-frit with hot-frit control.
5. Evaluate the surrogate; report where it fails.
6. Compare optimizer designs with baselines; verify each with the full solver.
7. **Tier 3** Repeat key results across parameter ranges to see which conclusions hold.

10.3 Limitations to state openly

- All results are model predictions, not measurements.
- The parallel-passage model has no cross-flow or heat conduction between slices; real beds have both, and randomized flow paths matter.
- Frits are lumped resistances; heat transfer uses a general packed-bed correlation.
- Gas flow is treated as adjusting instantly; reactor power is fixed unless Tier 3 coupling is added.
- Results depend on parameter values that carry uncertainty.

11 Pitch, Demo, and Q&A

11.1 Three-minute demo script

Time	What to say and do
0:00–0:20	**Hook.** One-sentence pitch; point to the feedback-loop diagram.
0:20–0:50	**The model.** "This is one fuel element split into 20 flow paths. It uses real hydrogen properties and checks itself against NASA's results."
0:50–1:30	**Break it.** Judge sets no cold frit and high power. The solver runs live; a hot spot runs away on screen.
1:30–2:00	**Fix it.** Judge adds frit resistance; same power is stable. Show the AI's instant prediction beside the solver's answer.
2:00–2:30	**AI design.** Switch to the AI-designed frit: higher stable power with less pressure drop than a uniform frit. Show the validation tab.
2:30–2:50	**Why it matters** and the main limitation.
2:50–3:00	**Close.** "We tested NASA's fix in a model you can break yourself, and the AI found a better frit."

11.2 Prepared answers to hard questions

Question	Honest answer
Isn't this just a simulation?	Yes, and most published particle bed stability research is computational. What matters is that it is verified, validated against independent results, and lets you pick the inputs.
How do you know it's right?	Verification tests show the code solves the equations; the validation tab shows how it compares with NASA, Tsinghua, and a textbook criterion, including any mismatch.
Did you tune it to match NASA?	No. Parameters came from sources and were committed to Git before validation runs; you can check the timestamps.
What does the AI add?	It predicts stability instantly and searches thousands of frit designs, which would be too slow with the solver alone. Its best designs are checked with the solver.
Does this apply to a real reactor?	It is an early-design and teaching tool. Real beds have cross-flow and other effects this model leaves out.
What did you build this weekend?	Point to the Git history and tier checklist.

11.3 Devpost checklist

- ☐ Title, tagline, one-sentence pitch.
- ☐ Model diagram and feedback-loop diagram.
- ☐ Demo video (under 2 minutes) showing a runaway, a fix, and the AI design.
- ☐ "Why this is not hardcoded" section and validation results.
- ☐ Limitations.
- ☐ Repository link with README and instructions to run the app.

12 Conclusions (To Be Written)

Write this section only after results exist. It should answer:

- Did the model reproduce the NASA and Tsinghua conclusions without tuning?
- How much frit resistance was needed for stability, and on which frit?
- How accurate was the surrogate, and where did it fail?
- Did the AI-designed frit beat the baselines, and by how much?
- Which limitations matter most?

12.1 Future work

- Cross-flow and conduction between slices.
- Comparison with CFD studies of the same geometry.
- Experimental validation with a cold-flow testbed or a hardware-in-the-loop rig.
- Full neutronics coupling.

13 Full Pipeline

> Check the SwampHacks XII rules at `https://xii.swamphacks.com`. Most hackathons require project code to be written during the event; planning, reading, and collecting parameters beforehand is usually fine.

Before the event

1. Read the NASA and Tsinghua papers and the SNTP report.
2. Collect every parameter with its source and range.
3. Write the validation agreement criteria.

4. Install Python, CoolProp, scikit-learn, and Streamlit; confirm they work.
5. Practice a small Streamlit app and a `scipy.optimize.root` example (not project code).
6. Ask UF Nuclear Engineering faculty to review the model plan.

During the event

Follow the time budget in Section 6: freeze parameters, build the solver, verify, run the stability studies, validate, build the app, finish Tier 1 by hour 24, then the AI layer, feature freeze at hour 31, submission, and rehearsal.

Sources

[1] NASA Lewis Research Center. *Flow instability in particle-bed nuclear reactors.* Nuclear Propulsion Technical Interchange Meeting. https://ntrs.nasa.gov/citations/19930017745

[2] Tsinghua University, Institute of Nuclear and New Energy Technology. *Analysis of the flow distribution in a particle bed reactor for nuclear thermal propulsion.* Energies 12(19):3590, 2019. https://www.mdpi.com/1996-1073/12/19/3590

[3] Y. Ji, Z. Li, J. Sun, E. You, M. Lang, L. Shi. *Numerical investigation and parametric study on thermal-hydraulic characteristics of particle bed reactors for nuclear thermal propulsion.* Nuclear Technology 206(8):1155–1170, 2020. doi:10.1080/00295450.2020.1760703

[4] NASA. *Deleterious thermal effects due to randomized flow paths in pebble bed and particle bed style reactors.* NTRS 20130011256

[5] *Space Nuclear Thermal Propulsion Program Final Report.* DTIC ADA305996

[6] Y. Chen (Tsinghua University). *Study of flow instability in particle bed reactors and analysis of mitigation measures.* 2025 doctoral forum abstract.

[7] S. Ergun. *Fluid flow through packed columns.* Chemical Engineering Progress 48:89–94, 1952.

[8] N. Wakao and S. Kaguei. *Heat and Mass Transfer in Packed Beds.* Gordon and Breach, 1982.

[9] M. Ledinegg. *Instability of flow during natural and forced circulation.* Die Wärme 61(8), 1938.

[10] I. H. Bell et al. *CoolProp: open-source thermophysical property library.* http://www.coolprop.org

[11] NIST Chemistry WebBook, thermophysical properties of fluid systems. https://webbook.nist.gov/chemistry/fluid/

[12] scikit-learn developers. *Gaussian processes.* https://scikit-learn.org

[13] SwampHacks XII. https://xii.swamphacks.com

[TO DO: Add the Bussard and DeLauer parallel-passage analysis cited in [1], the source for each parameter, and a reactor physics textbook if Tier 3 point kinetics is added.]

Acknowledgements

[TO DO: Thank SwampHacks XII organizers and sponsors, any UF faculty or mentors who reviewed the model, and anyone else who helped.]